

Relatório Técnico – Galactic Defender

Autores: Luis Gustavo Theml Novais e Mayara Mikaelly de Paula

Disciplina: Desenvolvimento de Jogos Digitais

1. Objetivo do Projeto

O objetivo deste projeto foi desenvolver um jogo 2D em HTML5 Canvas, no estilo nave espacial, explorando conceitos de animação, colisão, paralaxe, spritesheet e controle de teclado.

O jogo foi criado de forma incremental com o auxílio de prompts de IA, de modo a compreender a estruturação e evolução de um jogo do zero até a implementação de mecânicas mais complexas.

2. Descrição Geral do Funcionamento

O jogo apresenta:

- Uma nave controlada pelo jogador, com movimentação nas direções (WASD ou setas).
- Cenário com paralaxe, simulando profundidade no espaço.
- Inimigos com spritesheets animadas, que se alteram conforme o nível.
- Sistema de disparos: o jogador pode atirar para eliminar inimigos.
- Sistema de colisão AABB (Axis-Aligned Bounding Box).
- Sistema de vidas: o jogador começa com 3 vidas e perde uma a cada colisão com tiros inimigos.
- Quando as vidas chegam a zero, é exibida a mensagem "GAME OVER".

3. Estrutura de Arquivos do Projeto

```
|— index.html → Estrutura principal do jogo (Canvas + scripts)
|— style.css  → Estilos visuais básicos e centralização da tela
|— main.js   → Lógica do jogo (movimentação, colisão, animações)
```

4. Arquitetura e Detalhamento do Código

index.html

- Cria a estrutura base do jogo com uma tag **<canvas>** centralizada.
- Importa os arquivos **style.css** e **main.js**.
- Define título e metadados básicos.

Exemplo simplificado:

```
<canvas id="gameCanvas" width="800" height="600"></canvas>  
<script src="main.js"></script>
```

style.css

- Define centralização da tela, fonte legível, e borda do Canvas.
- Exemplo:

```
canvas {  
  border: 2px solid #000;  
  display: block;  
  margin: 40px auto;  
}
```

main.js

Contém toda a lógica do jogo, incluindo:

a) Inicialização e loop principal

- Criação do contexto 2D (**ctx = canvas.getContext('2d')**).
- Uso do **requestAnimationFrame()** para criar o loop de animação contínuo.
- Funções:
 - **update()**: atualiza posições, colisões e estados.

- `draw()`: desenha todos os elementos na tela.

b) Controle de teclado

- Mapeamento das teclas `W`, `A`, `S`, `D` e `↑`, `↓`, `←`, `→`.
- Armazenamento de estados de tecla pressionada e liberada.

c) Sistema de paralaxe

- Criação de camadas de fundo com velocidades diferentes, simulando profundidade.
- Exemplo:
 - Camada de estrelas (mais lenta)
 - Camada de nebulosas (média)
 - Camada de planetas (rápida)

d) Entidades

- Player (nave): possui posição, velocidade e método de disparo.
- Inimigos: se movem e disparam tiros periodicamente.
- Balas: projetis criados a partir da posição do player e inimigos.

e) Colisões

- Implementação do sistema AABB, verificando interseção entre retângulos de player, tiros e inimigos.

f) Sistema de vidas e Game Over

- O jogador inicia com 3 vidas.
- Cada colisão com um tiro inimigo reduz uma vida.

Ao zerar as vidas, o jogo exibe na tela:

GAME OVER!

-

g) Spritesheet e animações

Utiliza `drawImage()` com parâmetros de clipping para exibir o frame correto do sprite:

`ctx.drawImage(img, sx, sy, sw, sh, dx, dy, dw, dh);`

-

- Controle de quadros com `frameTimer` e `frameInterval` para regular a taxa de animação.

5. Mecânicas e Lógicas Implementadas

Mecânica	Descrição
Movimentação	Controlada por teclado com suavidade (WASD ou setas).
Disparo	Player dispara projéteis que colidem com inimigos.
Paralaxe	Fundo animado com camadas independentes e velocidades diferentes.
Animação via Spritesheet	Player e inimigos animados com frames extraídos via clipping.
Colisão AABB	Detecta sobreposição de retângulos entre player, inimigos e tiros.
Sistema de vidas	Player tem 3 vidas e o jogo termina ao perder todas.
Game Over	Mensagem exibida ao final, encerrando o loop principal.

6. Evolução do Projeto

De acordo com o arquivo `prompts utilizadas.txt`, a evolução seguiu esta sequência:

1. Criação das ideias iniciais
→ Prompt:
“Atue como desenvolvedor de jogos 2D em HTML5 Canvas. Sugira 3 ideias

de jogo no tema nave espacial...”

- Resultado: definição do tema, mecânicas e entidades básicas.

2. Construção do projeto base

→ Prompt:

“Agora com a 1ª ideia, crie um projeto base de jogo 2D em Canvas com...”

- Resultado: estrutura HTML, CSS e JS com loop, update/draw, controle e paralaxe.

3. Adição da animação do player (spritesheet)

→ Prompt:

“Dado o spritesheet com 36x48, implemente a animação do player...”

- Resultado: frames animados conforme estado (idle, run, shoot).

4. Implementação de inimigos com spritesheets diferentes por fase

→ Prompt:

“Implemente os spritesheets dos inimigos, alternando no jogo conforme o nível...”

5. Adição do sistema de vidas e Game Over

→ Prompt:

“Implemente a colisão do player com o tiro do inimigo, adicionando 3 vidas...”

Com isso, o jogo evoluiu de um protótipo básico para um jogo funcional completo, incluindo elementos de jogabilidade, animação e progressão.

7. Testes Realizados

- Teste de movimentação: verificou-se fluidez e resposta das teclas.
- Teste de colisão: checado se as balas e inimigos são detectados corretamente.
- Teste de animação: frames do spritesheet trocam conforme esperado.
- Teste de paralaxe: camadas se movem em velocidades diferentes.

- Teste de Game Over: confirmada a exibição após três colisões.

8. Prompts Utilizados

"Atue como desenvolvedor de jogos 2D em HTML5 Canvas. Sugira 3 ideias de jogo no tema nave espacial. cada uma com:

mecânicas centrais, entidades, estados do jogo, eventos de teclado, uso de paralaxe, colisão, spritesheet/clipping e disparo.

Inclua uma lista de tarefas priorizadas (MVP → Polimento)."

"Agora com a 1° ideia, crie um projeto base de jogo 2D em Canvas com:

index.html simples;

style.css básico (centralização e fonte legível);

main.js contendo:

loop de animação com requestAnimationFrame,

funções update() e draw(),

controle de teclado (WASD/setas),

cenário com 2–3 camadas de paralaxe,

estrutura de entidades (player, inimigos, balas),

sistema de colisão AABB.

Inclua comentários explicativos em cada seção do código."

"Dado o spritesheet com 36x48,

Implemente a animação do player (idle, run, shoot) no Canvas.

Utilize drawImage(img, sx, sy, sw, sh, dx, dy, dw, dh) para realizar o clipping.

Adicione controle de taxa de quadros (frameTimer)."

"Implemente os spritesheets dos inimigos, alternando no jogo conforme o nível, para simular a troca de fases."

"Implemente a colisão do player com o tiro do inimigo, adicionando 3 vidas para o player se o player tomar 3 tiros sua vida é zerada e o jogo da GAME OVER!"

9. Conclusão

O projeto foi bem-sucedido em aplicar conceitos fundamentais de desenvolvimento de jogos 2D utilizando HTML5, CSS e JavaScript. Através do uso de IA generativa, foi possível criar, aprimorar e refinar o código de forma iterativa, resultando em um produto final funcional, modular e facilmente extensível.